```
def qfunc(s,theta,layers=[(4,20),(20,2)],afn=torch.tanh):
    l1n = layers[0]
    l1s = np.prod(l1n)                          1
    theta_1 = theta[0:l1s].reshape(l1n)         2
    l2n = layers[1]
    l2s = np.prod(l2n)
    theta_2 = theta[l1s:l2s+l1s].reshape(l2n)
    bias = torch.ones((1,theta_1.shape[1]))
    l1 = s @ theta_1 + bias                     3
    l1 = torch.nn.functional.elu(l1)
    l2 = afn(l1 @ theta_2)                      4
    return l2.flatten()
```

- ***1*** Takes the first tuple in layers and multiplies those numbers to get the subset of the theta vector to use as the first layer
- ***2*** Reshapes the theta vector subset into a matrix for use as the first layer of the neural network
- ***3*** This is the first layer computation. The s input is a joint-action vector of dimensions (4,1).
- ***4*** We can also input an activation function to use for the last layer; the default is tanh since our reward ranges [-1,1].

This is the Q function implemented as simple 2-layer neural network (figure 9.13). It expects a state vector, `s`, that is the binary vector of neighbors states, and a parameter vector, theta. It also needs the keyword parameter, `layers`, which is a list of the form `[(s1,s2),(s3,s4)...]` that indicates the shape of the parameter matrix for each layer. All Q functions return Q values for each possible action; in this case they are for down or up (two actions). For example, it might return the vector `[-1,1]`, indicating the expected reward for changing the spin to down is –1 and the expected reward for changing the spin to up is +1.

Figure 9.13. The *Q* function for agent *j* accepts a parameter vector and a onehot encoded joint-action vector for the neighbors of agent *j*.

